

IDS

CODE:

```
import random
from collections import deque

N = 3

class PuzzleState:
    def __init__(self, board, x, y, depth, parent=None):
        self.board = board
        self.x = x
        self.y = y
        self.depth = depth
        self.parent = parent

    def __eq__(self, other):
        return isinstance(other, PuzzleState) and self.board ==
other.board

    def __hash__(self):
        return hash(tuple(map(tuple, self.board)))

row_delta = [0, 0, -1, 1]
col_delta = [-1, 1, 0, 0]

def is_goal_state(board):
    goal = [[1, 2, 3], [8, 0, 4], [7, 6, 5]]
    return board == goal

def is_valid(x, y):
    return 0 <= x < N and 0 <= y < N

def print_board(board):
    for r in board:
        print(' '.join(map(str, r)))
    print("-----")

def reconstruct_path(state):
    path = []
    while state:
        path.append(state.board)
        state = state.parent
    return path[::-1]
```

```

def depth_limited_search(state, limit, visited_for_current_dls):
    if is_goal_state(state.board):
        return state

    if state.depth == limit:
        return 'cutoff'

    visited_for_current_dls.add(state)

    cutoff_occurred = False

    for i in range(4):
        new_x = state.x + row_delta[i]
        new_y = state.y + col_delta[i]

        if is_valid(new_x, new_y):
            new_board = [r[:] for r in state.board]
            new_board[state.x][state.y], new_board[new_x][new_y] =
new_board[new_x][new_y], new_board[state.x][state.y]

            next_state = PuzzleState(new_board, new_x, new_y,
state.depth + 1, state)

            if next_state not in visited_for_current_dls:
                result = depth_limited_search(next_state, limit,
visited_for_current_dls)

                if result == 'cutoff':
                    cutoff_occurred = True
                elif result != 'failure':
                    return result

    return 'cutoff' if cutoff_occurred else 'failure'

def solve_puzzle_iddfs(start_board, x, y):
    initial_state = PuzzleState(start_board, x, y, 0)

    if is_goal_state(initial_state.board):
        print('Initial state is the goal state.')
        print_board(initial_state.board)
        return

```

```

max_search_depth = 50

for depth_limit in range(max_search_depth):
    print(f"Trying depth limit: {depth_limit}")
    visited_for_current_dls = set()

    result = depth_limited_search(initial_state, depth_limit,
visited_for_current_dls)

    if result != 'cutoff' and result != 'failure':
        print(f'Goal state reached at depth {result.depth}')
        print('Path to solution:')
        path = reconstruct_path(result)
        for i, board_step in enumerate(path):
            print(f"Step {i}:")
            print_board(board_step)
        return

    if result == 'failure':
        print(f'No solution found within depth {depth_limit}.')
        return

print(f'No solution found within maximum depth limit of
{max_search_depth}.')

if __name__ == '__main__':
    start_board = [[2, 8, 3], [1, 6, 4], [7, 0, 5]]

    blank_x, blank_y = -1, -1
    for r_idx in range(N):
        for c_idx in range(N):
            if start_board[r_idx][c_idx] == 0:
                blank_x, blank_y = r_idx, c_idx
                break
    if blank_x != -1:
        break

    print('Initial State:')
    print_board(start_board)

    solve_puzzle_iddfs(start_board, blank_x, blank_y)

```

OUTPUT :

```
... Initial State:
  2 8 3
  1 6 4
  7 0 5
-----
Trying depth limit: 0
Trying depth limit: 1
Trying depth limit: 2
Trying depth limit: 3
Trying depth limit: 4
Trying depth limit: 5
Goal state reached at depth 5
Path to solution:
Step 0:
  2 8 3
  1 6 4
  7 0 5
-----
Step 1:
  2 8 3
  1 0 4
  7 6 5
-----
Step 2:
  2 0 3
  1 8 4
  7 6 5
-----
Step 3:
  0 2 3
  1 8 4
  7 6 5
-----
Step 4:
  1 2 3
  0 8 4
  7 6 5
-----
Step 5:
  1 2 3
  8 0 4
  7 6 5
-----
```